

lb.js

ort

OU

 $S \setminus A$

wat

wat

ed

JS export.js X



middleware > JS export.js > HttpError

```
1 class HttpError extends Error {  
2   constructor(message, statusCode){  
3     super(message);  
4     this.statusCode = statusCode;  
5     this.name = "HttpError";  
6   }  
7 }  
8  
9 export default HttpError;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

node + - ... |

PS C:\Users\ANKIT MOR\OneDrive\Desktop\ANKIT\Node.js\employeeManagementSystem> npm run dev

```
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node server.js`  
db connected successfully  
Server has loaded on port 5000  
█
```

JS server.js ×

JS server.js > ...

```
1 import express from "express";
2 import HttpError from "../middleware/export.js";
3 import connectDB from "../config/db.js";
4 import employeeRoutes from "../routes/employeeRoutes.js";
5
6 const app = express();
7
8 app.use (express.json());
9
10 app.use("/Employee", employeeRoutes);
11
12 app.get("/", (req, res, next) => {
13   res.send("hello from server");
14 });
15
16 app.use((req, res, next) => {
17   return next (new HttpError("Not Found", 404));
18 });
19
20 app.use((error, req, res, next) => {
21   if (res.headersSent) {
22     return next(error);
23   }
24
25   res.status(error.statusCode || 500).json({
26     message: error.message || "internal server error",
27   });
28 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

node + v [] [] ... [] [] ×

PS C:\Users\ANKIT MOR\OneDrive\Desktop\ANKIT\Node.js\employeeManagementSystem> npm run dev

```
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
db connected successfully
Server has loaded on port 5000
[ ]
```

```
24
25   res.status(error.statusCode || 500).json({
26     message: error.message || "internal server error",
27   });
28 });
29
30 const port = 5000;
31
32 async function StartServer(){
33   try{
34
35     await connectDB();
36
37     app.listen(port,()=>{
38       console.log(`Server has loaded on port ${port}`);
39     });
40   }catch(error){
41     console.log(error.message);
42     process.exit(1);
43   }
44 }
45
46 StartServer();
47
```

> ...

```
1 import mongoose from "mongoose";
2
3 const employeeSchema = new mongoose.Schema({
4   name: {
5     type: String,
6     required: true,
7     trim: true
8   },
9   email: {
10    type: String,
11    required: true,
12    unique: true,
13  },
14  position: {
15    type: String,
16    required: true,
17  },
18  department: {
19    type: String,
20    required: true,
21  },
22  salary: {
23    type: Number,
24    required: true,
25  },
26
27 });
28
29
30 const Employee = mongoose.model("Employee", employeeSchema);
31
32 export default Employee;
```

JS employeeController.js X

controller > JS employeeController.js > ...

```
1 import Employee from "../modal/employee.js";
2 import HttpError from "../middleware/export.js";
3
4
5 const add = async (req, res, next) => {
6   try {
7
8     const { name, email, position, department, salary } = req.body;
9
10    const newEmployee = new Employee({
11      name,
12      email,
13      position,
14      department,
15      salary
16    });
17
18    await newEmployee.save();
19
20    res.status(201).json({
21      success: true,
22      message: "employee added successfully", newEmployee,
23    });
24  } catch (error) {
25    return next(new HttpError("employee not added", 500));
26  }
27 }
28
```

JS employeeController.js X

controller > JS employeeController.js > ...

```
29 const getAllEmployee = async (req,res,next)=>{
31
32     const employees =await Employee.find();
33
34     if(employees.length === 0){
35         return next (new HttpError("employee not found", 404));
36     }
37
38     res.status(200).json({
39         success:true,
40         message:"employee data found", employees,
41     });
42 }catch(error){
43     return next(new HttpError("employee data not found", 500));
44 }
45 }
46
47 const getEmployeeById = async (req,res,next)=>{
48     try{
49         const {id} = req.params;
50
51         const employeeId = await Employee.findById(id);
52
53         console.log(employeeId);
54
55         if(!employeeId){
56             return next (new HttpError("employee Id not found", 404));
57         }
58
59         res.status(200).json({
60             success:true,
61             message:"employee id found",employeeId,
62         });
63     }catch(error){
64         return next (new HttpError("employee id not found ",500));
65     }
66 }
```

JS employeeController.js X

controller > JS employeeController.js > ...

```
68 const updateEmployee = async (req, res, next) => {
69   const {id} = req.params;
70
71   const employeeUpdate = await Employee.findByIdAndUpdate(id, req.body, {new: true});
72
73   console.log(employeeUpdate);
74
75   if(!employeeUpdate){
76     return next(new HttpError("employee not updated", 400));
77   }
78   res.status(200).json({
79     success: true,
80     message: "employee update successfully", employeeUpdate,
81   });
82 }
83
84 catch(error){
85   return next(new HttpError("employee update not found", 500));
86 }
87
88
89 const updateEmployeeManually = async (req, res, next) => {
90   try{
91     const {id} = req.params;
92
93     const employeeUpdate = await Employee.findById(id);
94
95     if(!employeeUpdate){
96       return next(new HttpError("employee not updated", 400));
97     }
98
99     const updates = Object.keys(req.body);
100
101     const allowedUpdates = ["name", "email"];
102
103     const isValidOperation = updates.every(update => allowedUpdates.includes(update));
104   }
```



```

105  ✓    if(!isValidOperation){
106      |        return next(new HttpError("Invalid update fields", 400));
107      |    }
108
109      updates.forEach(update => employeeUpdate[update] = req.body[update]);
110
111      await employeeUpdate.save();
112
113  ✓    res.status(200).json({
114      |        success:true,
115      |        message:"employee update successFully", employeeUpdate,
116      |    });
117
118  ✓    }catch(error){
119      |        return next (new HttpError("employee update not found",500));
120      |    }
121  }
122
123  ✓  const deleteEmployee = async (req, res, next) => {
124  ✓  try {
125      |      let { id } = req.params;
126
127      |      id = id.trim();
128
129      |      console.log("REQ PARAM ID =>", id);
130
131      |      const checkEmployee = await Employee.findById(id);
132      |      console.log("FOUND BEFORE DELETE =>", checkEmployee);
133
134  ✓  |      if (!checkEmployee) {
135      |      |          return next(new HttpError("employee id not found", 404));
136      |      |      }
137
138      |      await Employee.findByIdAndDelete(id);

```

```
139
140 ✓ res.status(200).json({
141     success: true,
142     message: "employee deleted successfully"
143 });
144
145 ✓ } catch (error) {
146     console.log("DELETE ERROR =>", error);
147     return next(new HttpError(error.message, 500));
148 }
149 };
150
151 ✓ const deleteAllEmployee = async (req,res,next)=>{
152     try{
153
154         const result = await Employee.deleteMany({});
155
156         if (result.deletedCount === 0) {
157             return next(new HttpError("No employees found", 404));
158         }
159
160         res.status(200).json({
161             success:true,
162             message:"All employee deleted successfully", deletedCount: result.deletedCount,
163         });
164     }catch(error){
165         return next (new HttpError("employee id not found ",500));
166     }
167 }
168
169 ✓ export default {
170     add,
171     getAllEmployee,
172     getEmployeeById,
173     updateEmployee,
174     updateEmployeeManually,
```

```
routes > JS employeeRoutes.js > ...
```

COLLECTIONS

My Collection

employeeManagementSystem

check server

added

AllEmployee

EmployeeFindById

updateEmployee

deleteEmployeeById

deleteAllEmployee

updateManully

event_management_system

StudentManagementSystem

ENVIRONMENTS

SPECS

FLAWS

POST

http://localhost:5000/Employee/add

Send

Docs

Params

Authorization

Headers

Body

Scripts

Tests

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

```
1 {
2   "name": "ankit4",
3   "email": "ankit4@123gmail.com",
4   "position": "HR",
5   "department": "software development",
6   "salary": 49000
7 }
```

Body

Cookies

Headers

Test Results

201 Created

45 ms

467 B

Save Response

JSON

Preview

Visualize

```
1 {
2   "success": true,
3   "message": "employee added successfully",
4   "newEmployee": {
5     "name": "ankit4",
6     "email": "ankit4@123gmail.com",
7     "position": "HR",
8     "department": "software development",
9     "salary": 49000,
10    "_id": "6a17fe05d616016a0f634b279",
11    "__v": 0
12  }
13 }
```